

第 1 篇 大模型視野、數學與工程基礎

Vision, Mathematics, and Engineering Foundations | 第 1–4 章

本篇承諾：建立共同語言、數學直覺與可重現的開發環境。高中生能看懂全貌，大學生能推導公式，工程學生能在筆電上跑出第一個實驗。

第 1 章

大模型全貌、能力邊界與臺灣機會

The LLM Landscape, Capability Boundaries, and Taiwan Opportunity

難度：核心 | 建議授課：第 1 週 | 硬體需求：A 級（一般筆電即可完成全部實作）

叫獸開講

一位雲林的高中老師打開某個知名的聊天機器人，問它「我們學校今年免試入學的超額比序怎麼算」。模型答得頭頭是道，條理分明，語氣自信——然後把新北市的規則整套搬了過來。它不是不會講中文，它是不認識臺灣。這一章要回答的問題是：為什麼會這樣？以及，我們能不能做出一個真的認識這裡的模型？

學習目標

讀完本章並完成三個程式實作與一個系統案例後，你應該能夠：

1. 用一張同心圓圖區分人工智慧、機器學習、深度學習、生成式 AI、基礎模型、大語言模型與推理模型，並說出每一層的判準。
2. 列舉大模型的七類核心能力，並為每一類找出至少一個臺灣本地的應用情境。
3. 說明幻覺、知識時效、偏見、脆弱推理與過度自信五種失效模式的成因，並指出各自對應的工程對策。
4. 比較封閉 API、開放權重與自建模型三條路線在控制力、成本、隱私、授權與能力上限上的差異，並為一個給定需求做出選擇。
5. 畫出大模型的七階段生命週期，指出每階段的產出物、負責角色與最容易被跳過的一步。
6. 用可查證的證據（用語、制度知識、token 效率）論證臺灣為何需要本土模型，而不是只靠翻譯或提示工程。
7. 實際量測三個模型在同一組繁體中文提示上的正確性、用語、token 數、延遲與成本，並做出可解釋的比較圖表。
8. 完成一份「臺灣本土大模型需求規格書 v0.1」，明確定義使用者、任務、語言、資料邊界、評測

指標與不可接受風險。

給高中讀者的提示

本章沒有任何需要微積分或線性代數的內容，只有觀念與一個要動手跑的實驗。1.1 到 1.3 節請務必讀完，1.4 與 1.5 節可以先讀表格、跳過細節，等到第 22 章要真正選模型時再回來看。

1.1 人工智慧、生成式 AI、基礎模型、大語言模型與推理模型的關係

這幾年，「AI」這兩個字被用得太浮濫，浮濫到它幾乎失去了資訊量。有人說「我們公司導入了 AI」，可能是指買了一套規則引擎；也可能是指工程師寫了一支腳本去呼叫某個聊天機器人的 API。在課堂上，我們必須先把名詞的邊界劃清楚，否則後面四十章的討論會一路模糊下去。

1.1.1 一組同心圓，但不是每一圈都真的包含

最常見的教學方式是畫同心圓：人工智慧最大，裡面包著機器學習，再裡面是深度學習，然後是生成式 AI，最裡面是大語言模型。這張圖大致正確，但有兩個地方容易誤導初學者，必須先講清楚。

第一，人工智慧不等於機器學習。早期的專家系統、規則引擎、搜尋演算法、邏輯推論器都是不折不扣的人工智慧，它們沒有任何「學習」的成分。工廠裡跑了二十年的排程最佳化程式、下棋程式裡的 alpha-beta 剪枝、導航軟體裡的 A* 搜尋，全都是 AI。把 AI 直接等同於「用資料訓練出來的模型」，會讓你看不見這些方法在今天仍然有效——事實上，第 32 章講 Agent 時，我們會回頭把規則與搜尋當成安全機制的一部分。

第二，生成式 AI 不完全被深度學習包含。統計式的馬可夫鏈文字產生器、文法式的模板生成器都會「生成」內容，只是效果差。今天我們談的生成式 AI 幾乎都是深度學習做的，但概念上兩者不是嚴格的包含關係。

所以更精確的說法是：這是一組「大致由外而內、但邊界會互相滲透」的概念層。我們用下表把每一層的判準寫死，往後全書都以此為準。

層級	判準（怎麼判斷屬不屬於）	代表方法	在本書的位置
人工智慧 AI	讓機器展現通常需要人類智能才能完成的行為，不限手段	規則引擎、搜尋、規劃、機器學習	第 32、37 章（安全與規則約束）
機器學習 ML	行為來自資料中歸納出的參數，而非人工撰寫的規則	迴歸、決策樹、SVM、神經網路	第 3 章（最佳化基礎）
深度學習 DL	使用多層神經網路自動學出表徵，不	CNN、RNN、Transformer	第 10–13 章

層級	判準（怎麼判斷屬不屬於）	代表方法	在本書的位置
	需人工設計特徵		
生成式 AI	輸出是高維度的新內容（文字、影像、語音），而非單一標籤或數值	GPT 系列、擴散模型、語音合成	全書主線
基礎模型 FM	在大規模廣泛資料上預訓練、可經少量調整適配到大量下游任務	GPT、Llama 系列、CLIP、Whisper	第 22–26 章（適配）
大語言模型 LLM	以文字（詞元序列）為主要模態的基礎模型，通常是自回歸解碼器	Decoder-only Transformer	第 11–20 章
推理模型	在輸出答案前會產生較長的內部思考過程，並以此換取正確率	思考鏈訓練、可驗證獎勵強化學習	第 27 章
多模態模型 VLM/VLA	同時處理文字與影像、語音或動作，共用同一個語言骨幹	視覺編碼器 + 語言模型	第 38–39 章

1.1.2 「大」到底大在哪裡

大語言模型的「大」，指的其實是三件同時發生的事，缺一不可：

- 參數量大。從早期的數億到今天動輒數百億、數千億。參數量決定模型能記住多少東西、能表達多複雜的函數。
- 訓練資料大。以兆 (trillion) 為單位的詞元數。資料量決定模型見過多少世界。
- 訓練算力大。以 GPU 年為單位的浮點運算。算力是前兩者能不能被真正吸收的前提。

這三者不是各自獨立可以任意加大的旋鈕，它們之間有相當精確的最佳配比關係——第 17 章的尺度定律會用數學把這件事講清楚。現在你只要記住一個直覺：如果你有一台很大的模型卻只餵了很少的資料，那是浪費；如果你有海量資料卻用一個很小的模型去吃，那是吃不下。大模型工程的核心問題之一，就是在給定的算力預算下，決定模型該多大、資料該多少。

另外要提醒一個常見的誤會：參數量不是能力的唯一指標，甚至常常不是最重要的指標。一個經過良好資料清理、良好後訓練、良好對齊的 8B 模型，在特定任務上打敗一個粗製濫造的 70B 模型是家常便飯。第 26 章會用具體實驗證明這件事，而這正是臺灣的機會所在——我們不一定要比大，但我們可以比準、比合適。

1.1.3 基礎模型：一個 2021 年才被正式命名的概念

「基礎模型」(foundation model) 這個詞是史丹佛大學的研究群在 2021 年提出的，用來描述一種

新的產業結構：不再是每個任務訓練一個模型，而是先訓練一個通用的大模型，再由無數下游應用去適配它。這個轉變的意義比技術本身還大——它把 AI 從「每家公司各自訓練」變成「少數機構訓練、多數機構適配」，也因此帶來了集中化、依賴與主權的問題。這正是第 1.6 節與第 37 章要處理的核心議題。

基礎模型的三個特徵是：規模（scale）、通用性（generality）、可適配性（adaptability）。第三點最關鍵。如果一個模型很大很強，但每次要在新任務上都得從頭訓練，那它就不是基礎模型，只是一個大模型。正因為可適配，才有了第六篇整整五章在講的持續預訓練、指令微調、LoRA 與偏好對齊——這些技術的存在前提，就是有人先幫我們把基礎模型訓練好了。

1.1.4 指令模型與推理模型的分野

你在網頁上使用的聊天機器人，背後至少經過兩個階段：先在大量文本上預訓練成一個「會接話」的基礎模型，再經過指令微調與對齊，變成一個「會照你的話做事」的指令模型。這兩者的行為差異非常大。把一個純粹的基礎模型直接拿來聊天，你問它「臺灣的首都在哪裡」，它可能回你「臺灣的首都在哪裡？這是一個常見的地理問題。下一題：日本的首都在哪裡？」——因為它學到的是「文本接下去長什麼樣」，不是「回答問題」。第 23 章會讓你親眼看到這個差異。

推理模型則是更近期的分野。它在給出最終答案之前，會先產生一段（有時很長的）內部思考，把問題拆解、嘗試、檢查、修正。代價是輸出的詞元數暴增、延遲拉長、成本上升；報酬是在數學、程式、邏輯與多步驟規劃這類任務上有顯著的正确率提升。第 27 章會告訴你一件對臺灣特別重要的事：讓模型「想久一點」，往往比讓模型「變大一點」便宜得多——這對算力受限的我們是好消息。

1.1.5 為什麼贏的是 Transformer

學生常問：注意力機制在 2014 年就有了，RNN 也一直在改進，為什麼是 2017 年的 Transformer 改變了一切？答案不在「它比較聰明」，而在「它比較適合現在的硬體」。

RNN 有一個結構性的詛咒：要算第 t 個字，必須先算完第 $t-1$ 個字。這是本質上的序列相依，無論你買多少張 GPU 都無法平行化。而 GPU 的整個設計哲學就是「同時做非常多次相同的運算」。把 RNN 放上 GPU，就像請一支交響樂團排隊一個一個上台獨奏——樂器再多也沒用。

Transformer 把序列相依拆掉了。在訓練時，一整個句子的所有位置可以同時計算，注意力矩陣是一次矩陣乘法算出來的。這讓「訓練資料變多」這件事第一次真正變得划算——你加資料、加卡、加時間，模型就會變好，而且這個關係穩定到可以寫成公式（第 17 章）。這種「投入可預期地轉成能力」的性質，是資本願意大規模進場的前提，也是整個大模型時代的真正起點。

這件事對本書的讀者有一個很實際的啟示：架構的價值不能脫離硬體來談。第 20 章討論線性注意力與狀態空間模型時，我們會用同一把尺去量它們——不是問「理論上更優雅嗎」，而是問「在真實的 GPU 與真實的記憶體頻寬上，它跑得比較快嗎」。

1.1.6 名詞速記表

以下十個縮寫會貫穿全書，請先建立印象，細節在對應章節。

縮寫	全稱	一句話說明	詳見
LLM	Large Language Model	以文字為主要模態的大型基礎模型	第 11–20 章
SFT	Supervised Fine-Tuning	用「問題—標準回答」配對教模型聽話	第 23 章
PEFT	Parameter-Efficient Fine-Tuning	只訓練少量新增參數的微調技術總稱	第 24 章
LoRA	Low-Rank Adaptation	最常用的 PEFT 方法，用低秩矩陣近似權重更新	第 24 章
RLHF	Reinforcement Learning from Human Feedback	用人類偏好訓練模型的行為	第 25 章
DPO	Direct Preference Optimization	不需獎勵模型的直接偏好最佳化	第 25 章
RAG	Retrieval-Augmented Generation	先檢索再生成，讓答案可附出處	第 30–31 章
MoE	Mixture of Experts	每次只啟用部分專家，用稀疏換效率	第 20 章
VLM	Vision-Language Model	同時看得懂影像與文字的模型	第 38 章
SLO	Service Level Objective	你對服務品質（延遲、可用率）的量化承諾	第 35 章

常見誤解澄清

「大模型就是一個超大的資料庫，把網路上的東西存起來再查出來給你。」這是錯的。模型儲存的是參數，不是文本；它沒有一份可以查詢的原文副本。這也正是幻覺的來源之一：它記住的是統計上的關聯，不是逐字的事實。想要逐字可查、可附出處的答案，你需要的是第 30 章的 RAG，不是更大的模型。

1.2 大模型能做什麼：從生成到工具使用

要判斷一項技術值不值得投入，得先看清楚它的能力邊界。這一節先講「能做什麼」，下一節馬上講

「不能保證什麼」——這兩節必須連在一起讀，只讀一半的人不是過度樂觀就是過度悲觀，兩種都不適合做工程。

1.2.1 七類核心能力

能力類別	具體任務	臺灣情境範例	成熟度
生成	寫作、改寫、擴寫、風格轉換、創意發想	公文初稿、教案設計、行銷文案、學習單	高（但需人工把關）
理解與抽取	分類、命名實體辨識、關係抽取、意圖判讀	工單分類、履歷解析、法條要素抽取、客服意圖	高
翻譯與轉寫	跨語言翻譯、文白轉換、繁簡轉換、術語統一	技術文件中英互譯、臺語文字化、公文白話化	中高（術語需在地校正）
摘要	單文摘要、多文摘要、會議紀錄、重點條列	會議紀錄、法規修正對照、產線異常日報	高
程式	補全、生成、除錯、重構、測試撰寫、程式解釋	工控腳本、資料處理、教學範例、單元測試	中高（必須人工審查）
工具使用	函式呼叫、查資料庫、執行運算、操作外部系統	查詢校務系統、排課、下工單、控制機械手臂	中（風險最高）
多模態互動	看圖回答、文件影像理解、語音對話、動作生成	設備面板判讀、表單掃描、臺語語音助理、機器人指令	中（快速演進中）

1.2.2 為什麼「猜下一個字」可以做到這些事

這是每個學生第一次聽到大模型原理時都會問的問題：它只是在猜下一個詞元，怎麼可能會寫程式、會摘要、會翻譯？

一個有用的思考角度是：如果你要在所有可能的文本上，把「下一個字是什麼」猜得夠準，你被迫要學會很多東西。要猜對一段程式碼的下一行，你得學會語法與邏輯；要猜對一篇論文的結論，你得學會前面的推導；要猜對一段對話的回應，你得學會語境與意圖。換句話說，下一詞元預測是一個「表面上很簡單、但要做到極致就必須理解世界」的目標函數。第 5 章會用機率的語言把這句話說得更嚴謹。

但這個說法有它的極限，我們也必須誠實：模型學到的是「什麼樣的文字在統計上會接在後面」，這

與「什麼是真的」之間有一道鴻溝。大部分時候兩者高度重合——因為人類寫下來的文字大多是對的——但一旦進入長尾、進入冷門、進入模型沒見過的地方，這道鴻溝就會裂開，那就是幻覺。

1.2.3 能力的三個來源：預訓練、後訓練、推論期

學生常把模型的表現當成一個整體來看待，但工程上必須拆開，因為每一塊的改善方法完全不同、成本也差了好幾個數量級。

能力來源	決定了什麼	改善成本	對應章節
預訓練	世界知識、語言能力、基本推理、程式能力的天花板	極高（數十萬至數千萬元起跳）	第 9、13–20 章
後訓練	是否聽話、風格、拒答邊界、特定領域強度、推理習慣	中（單張 GPU 可做 LoRA）	第 22–27 章
推論期	這一次回答有沒有拿到對的資訊、有沒有用對工具、想得夠不夠久	低（改提示、加檢索、加工具）	第 27、29–32 章

1.2.4 怎麼實際量出「能力邊界」在哪裡

前面那張能力表寫著「成熟度：高」或「中」，這種說法對規劃有用，但對工程決策不夠。你真正需要知道的是：在你的任務上，這個模型從哪一點開始不可靠。以下是一個可以在一個下午內做完、卻能省下你數週摸索的方法，本書稱之為難度階梯法。

1. 把你的任務拆成一個由易到難的階梯，至少五階。例如摘要任務可以是：單段落摘要 → 單篇文章摘要 → 含表格的技術文件摘要 → 跨三份文件的比較摘要 → 需要判斷矛盾之處的多文件摘要。
2. 每一階出五到十題，難度必須真的遞增，而且要有明確的評分標準。
3. 對每一階量測正確率，畫出「難度—正確率」曲線。
4. 曲線開始明顯下墜的那一階，就是這個模型在你任務上的能力邊界。
5. 在邊界之外的任務，不要用提示工程硬撐——那是最常見的時間陷阱。應該改變系統設計：拆解成多步驟、加入檢索、加入驗證器，或換模型。

這個方法的價值在於它把「這個模型行不行」這種無法回答的問題，換成「它到第幾階開始不行」這種可以量測的問題。你會在第 16 章看到同樣的思路被用在實驗設計上，在第 36 章看到它被擴充成完整的評測分層。

大多數失敗的大模型專案，不是敗在選錯模型，而是敗在把任務設定在能力邊界之外，然後花三個月試圖用提示把它救回來。先量邊界、再設計系統，順序不能顛倒。

1.2.5 湧現能力：一個仍有爭議的概念

你會在很多科普文章裡看到「湧現」(emergence) 這個詞：模型小的時候某項能力完全不存在，參數量跨過某個門檻後突然出現。多步驟算術、複雜指令遵循、少樣本學習都被舉為例子。這個說法很有戲劇性，也確實描述了部分現象，但學界對它有相當有力的反駁，值得學生知道。

反駁的核心是：所謂的「突然出現」，有一部分是評測指標造成的錯覺。如果你用「答案完全正確才給分」這種全有全無的指標去量測一個需要五個步驟的任務，那麼模型從「五步錯三步」進步到「五步錯一步」時，分數依然是零；直到某一刻全部答對，分數才從零跳到一。看起來像突然湧現，實際上底層能力是平滑成長的。換成連續型指標（例如逐步驟正確率、對數機率），曲線往往就變得平滑了。

這件事對工程實務有兩個直接的教訓，兩個都會在第 36 章重演：第一，你選的指標會決定你看到的故事，所以評測設計必須在實驗開始之前就想清楚；第二，「突然變好」與「突然變壞」都要先懷疑是量測方式的問題，再懷疑是模型的問題。這是一種應該養成的專業反射。

至於湧現到底存不存在，本書的立場是：不必急著下結論，但務必用連續指標去看你自己的實驗。

這張表隱含一個對臺灣極為重要的策略判斷：預訓練那一欄我們短期內很難自己撐起來，但後訓練與推論期這兩欄，一個大學實驗室、一家中小企業、甚至一位認真的大學生就能做出真正的差異。本書把四十章裡的二十多章放在這兩欄，正是這個原因。

1.3 大模型不能保證什麼：五種失效模式

如果你只記得本章的一件事，我希望是這一節。系統上線後出事，幾乎不會是因為工程師不知道模型很強，而是因為他不知道模型會在哪裡出錯。

1.3.1 幻覺 (hallucination)

模型會用完全自信的語氣，說出一個不存在的法條條號、一篇不存在的論文、一個不存在的函式名稱。這不是偶發的 bug，而是訓練目標的直接後果：模型被獎勵的是「產生看起來合理的文字」，不是「產生真實的文字」。當它不知道答案時，它沒有「我不知道」這個預設選項，除非我們在後訓練階段教它。

幻覺最容易出現在三種地方：一是長尾知識（冷門的人、地、事、數字）；二是需要精確引用的內容（條號、頁碼、日期、型號）；三是使用者用強烈的預設立場逼問時（「請告訴我第 27 條寫了什麼」

——即使根本沒有第 27 條，模型也傾向給你一個）。

工程對策不是「叫模型不要亂講」，那沒有用。真正有效的是：把事實查詢外包給檢索系統並強制附上出處（第 30–31 章）、在後訓練時明確教會拒答（第 25 章）、以及在系統設計上讓不確定性可見（第 33 章）。

1.3.2 知識時效

模型的知識停在訓練資料的截止時間。校長換了、法規修了、課綱改了、產品換代了，模型不會知道。更麻煩的是它通常也不知道自己不知道——它會用舊資訊回答你，語氣一樣自信。這在校務、法規、醫療衛教、產品規格這類場域是致命的。

對策有三條路，成本由低到高：檢索增強（讓模型每次都去查最新文件）、模型編輯（第 28 章，動手改掉特定事實）、重新訓練（最貴，通常不划算）。實務上九成以上的情況應該選第一條。

1.3.3 偏見與代表性不足

模型的世界觀來自訓練語料的分布。如果語料裡簡體中文與中國語境的比例遠高於繁體中文與臺灣語境，模型就會在不自覺中把中國的制度、用語、地名當成預設值——本章開頭那位老師遇到的正是這件事的溫和版本。更嚴重的版本會出現在職業、性別、族群、地域的刻板連結上。

這裡要特別提醒一個容易被忽略的面向：臺灣的原住民族語、客語、臺語在網路語料中的比例極低，而這些語言恰恰是最需要 AI 協助保存的。第 39 章會把這件事當成一個具體的工程任務來處理，而不是只當成一句政治正確的口號。

1.3.4 脆弱推理

模型在標準格式的題目上表現優異，但把同一題換個講法、加一句無關的干擾條件、把數字改大一點，正確率就可能大幅下降。這說明它有相當一部分的「推理」其實是對訓練資料中解題模式的比對，而不是真正的邏輯演繹。

判斷方法很簡單，你在第 27 章會實際做一次：把題目的表面特徵改掉但保持邏輯結構不變，看正確率掉多少。掉得越多，代表越依賴表面模式。這也是為什麼推理模型與可驗證獎勵訓練會成為近年的重點——它們試圖讓模型的推理真的是推理。

1.3.5 過度自信與校準不良

一個校準良好的系統，說「我有七成把握」時，它應該有大約七成的時候是對的。大模型在這件事上表現不佳：它的語氣自信程度與實際正確率之間的相關性很弱，而且經過對齊訓練之後往往更糟——

因為人類標註者偏好聽起來有自信的回答。

這對系統設計的意涵是：不要相信模型自己說的信心程度，要用外部方法估計。可用的手段包括多次取樣看一致性（第 27 章）、檢索結果的支持度（第 30 章）、以及驗證器（第 27 章）。

1.3.6 兩個容易被忽略的附加問題

除了上述五種，還有兩種問題不常被列入「失效模式」，但在實務上造成的困擾一點也不少。

第一是不可重現性。只要解碼時使用了隨機取樣（第 14 章），同樣的輸入就會得到不同的輸出。這對展示無害，對驗收與除錯卻是災難：使用者回報的錯誤你可能重現不出來，昨天通過的測試今天可能失敗。工程上的處理方式是：在測試與驗收情境把 `temperature` 設為 0 或固定亂數種子，並在系統中記錄每一次呼叫的完整參數與模型版本。這件事必須從專案第一天就做，事後補做的成本極高。

第二是上下文限制與位置效應。模型的上下文長度有上限，超過就會被截斷——而且截斷通常是靜默的，不會報錯。更微妙的是，即使沒有超過上限，放在上下文中間的資訊也比放在開頭與結尾的資訊更容易被忽略，這個現象在研究上被稱為「迷失在中間」。這意味著你不能天真地把一百份文件全部塞進上下文就期待模型會全部讀進去。第 29 章的情境工程與第 30 章的檢索排序，處理的正是這個問題。

1.3.7 怎麼向非技術的決策者解釋這些風險

這是一項被嚴重低估的工程能力。你的校長、廠長、主任不會讀本書，但他們要決定要不要導入、要不要簽字、出事要不要負責。用「模型會產生幻覺因為它的訓練目標是最大化下一詞元的條件機率」去解釋，只會得到禮貌的點頭與錯誤的期待。

以下三組類比在實務上效果良好，你可以直接拿去用，但務必記得類比都會失真，關鍵處要補上正確的說法。

要解釋的事	可用的類比	類比失真之處（必須補充）	對應失效
為什麼會編造	像一位口才極好、讀過很多書但沒帶參考資料的顧問，被追問時不會說不知道	模型沒有「知道自己不知道」的內建機制，不是不願意承認	幻覺
為什麼會過期	像一本印刷精美的年鑑，內容停在出版那一年	它不會像書一樣標示出版年份，也不會提醒你	知識時效
為什麼要附出處	像醫師開藥要寫依據，不能只說「我的判斷」	模型的「出處」若非來自檢索，也可能是編的	幻覺、時效

要解釋的事	可用的類比	類比失真之處（必須補充）	對應失效
為什麼答案會不一樣	像請三位不同的助理寫同一份摘要	差異來自隨機取樣，可以用參數關掉	不可重現

在提案文件中，我建議永遠把三件事寫在第一頁：這個系統會在什麼情況下出錯、出錯時誰會發現、以及出錯時的補救成本。把風險寫在前面不會讓提案被否決，反而通常會讓決策者更願意簽字——因為他知道你是清醒的。

1.3.8 把失效模式整理成工程檢查表

失效模式	成因	可觀察徵兆	工程對策	章節
幻覺	訓練目標追求流暢而非真實； 長尾知識缺乏	編造條號、論文、函式名； 細節越具體越可疑	RAG 強制引用、拒答訓練、輸出驗證	25、30、31
知識時效	訓練資料有截止時間	使用舊人名、舊法規、舊版本	檢索增強、模型編輯、知識更新流程	28、30
偏見	語料分布不均；臺灣語料占比低	用語漂移、制度誤植、族群刻板連結	語料配比、本土評測、偏誤量測	9、22、36
脆弱推理	部分推理來自模式比對而非演繹	換句話說就答錯；加干擾條件正確率驟降	推理模型、驗證器、多樣本一致性	27
過度自信	對齊訓練偏好自信語氣；校準不佳	語氣自信但內容錯誤；信心與正確率不相關	外部信心估計、一致性檢查、人類覆核	27、30、37
不可重現	解碼使用隨機取樣	同輸入不同輸出；測試時好時壞	固定溫度與種子、記錄完整參數	14、16
上下文溢出	長度上限與位置效應	長文件後半段被忽略；靜默截斷	情境工程、檢索排序、長度預算	29、30

工程守則

本書從第 1 章到第 40 章會反覆出現同一句話：任何一個大模型系統，都必須先定義「不可接受的失敗」是什麼，再開始設計。校務助理答錯社團名稱是可容忍的，答錯申請截止日期不是；衛教助理講錯保健常識是可容忍的，給出用藥劑量建議不是。這條線畫在哪裡，是工程師與領域專家共同的責任，不是模型的責任。

1.4 從封閉 API、開放權重到自建模型：四條路線的取舍

假設你是某科技大學的教師，想做一個能回答校規與課程問題的助理。你有四條路可以走，而選錯路線的代價通常在半年後才會顯現。

1.4.1 四條路線

路線	做法	啟動成本	典型適用情境
封閉 API	呼叫商業模型的 API，加上提示與檢索	極低（數小時）	快速驗證概念、資料不敏感、量小
開放權重 + 提示 / RAG	自行部署開放權重模型，不改權重	低（數天）	資料不可外流、需控制成本、量中等
開放權重 + 適配	在開放權重模型上做持續預訓練或微調	中（數週至數月）	需要領域能力、繁中強化、長期使用
自建預訓練	從零訓練基礎模型	極高（千萬元級以上）	國家級主權需求、特殊模態、研究目的

1.4.2 六個必須同時評估的維度

1. 控制力：模型會不會在你不知情的時候被換掉、被下架、被改變行為？封閉 API 的答案是「會」，而且你通常不會收到事前通知。這對已經通過驗收的系統是重大風險。
2. 成本結構：API 是變動成本（用多少付多少），自建是固定成本（先投入硬體，之後邊際成本低）。哪個划算取決於用量。第 21 與 35 章會教你算出損益兩平點。
3. 隱私與資料落地：學生輔導紀錄、病歷、製程參數、設計圖能不能離開你的機房？這通常不是技術問題而是法規與契約問題，而且答案往往直接排除掉某些路線。
4. 授權合規：這是最多人踩雷的一格，下一小節專門講。
5. 可維護性：你的團隊有沒有人能在半夜三點處理 GPU 掉卡、模型服務崩潰、品質突然下降？沒有的話，自建的總持有成本會遠高於帳面數字。
6. 能力上限：目前最強的通用能力仍在少數封閉模型手上。但要注意——你需要的是「這個任務上夠好」，不是「全球最強」。第 36 章會教你怎麼量測「夠好」。

1.4.3 「開源」與「開放權重」不是同一件事

這是全書最重要的名詞澄清之一，請務必記住。

- 開放權重（open weights）：模型參數可以下載，但訓練資料、訓練程式碼、資料配方通常不公

開。你可以用，但你無法完整重現它，也很難確知它訓練時吃了什麼。

- 開源 (open source)：在軟體的傳統定義下，應該包含可重現所需的一切。真正符合這個標準的大模型很少。
- 授權條款各不相同：有的允許商用、有的限制月活躍用戶數、有的要求衍生模型必須標註來源、有的禁止用來訓練其他模型、有的對特定地區或用途有限制。

實務上請養成一個習慣：在下載任何模型權重之前，先把授權條款讀完，並把以下五個問題的答案寫進專案文件——能不能商用、能不能再散布、衍生模型有什麼義務、有沒有使用規模上限、有沒有禁止用途。第 22 章會提供一份完整的檢查表，附錄 F 則有可直接填寫的表格。

臺灣單位常見的三個誤區

第一，以為「可以下載」就等於「可以商用」——不一定。第二，以為用了開放權重模型就沒有法遵問題——你自己餵進去的資料仍然受個資法與契約拘束。第三，以為封閉 API 一定比較安全——資料是否被用於訓練、儲存在哪個司法管轄區、保留多久，都必須逐條確認，不能靠印象。

1.4.4 一個完整的成本試算範例

路線選擇最後常常會落到一個問題上：到底哪個比較便宜？學生的直覺通常是「自建一定比較貴」或「API 一定比較貴」，兩種直覺都不可靠。下面用一個具體的例子把帳算完，你在課後習題會用同樣的方法算你自己的專案。

情境：某科技大學的校務問答助理，日間使用為主，尖峰每秒約 3 次查詢，全日約 5,000 次查詢。每次查詢的提示（含檢索到的文件片段）約 1,800 個詞元，回覆約 400 個詞元。

項目	A 方案：商業 API	B 方案：地端開放權重	備註
一次性投入	0 元	約 45 萬元（單機雙卡與周邊）	含伺服器、UPS、機櫃
每月變動成本	約 4.0 萬元（依詞元計價）	約 1.2 萬元（電力 + 維運）	不含人力
第一年總成本	約 48 萬元	約 59 萬元	B 方案較高
第三年累計	約 144 萬元	約 88 萬元	B 方案較低
損益兩平	—	約第 17 個月	$(45) \div (4.0 - 1.2) \approx 16.1$ 個月

光看這張表，超過一年半的專案就該自建。但這個結論太快了，因為表裡漏掉三項通常更重要的成本：

- 人力。地端方案需要有人處理 GPU 驅動、模型更新、服務崩潰與監控告警。若換算成 0.2 個人力，每月至少再加兩萬元以上，損益兩平會延後到三年以外。
- 機會成本。自建的建置期可能要一到三個月，這段期間沒有服務可用；API 方案當天就能上線。對需要快速驗證的專案，這段時間比錢貴。
- 品質差距。若地端模型在關鍵任務上的正確率低 8 個百分點，你省下的錢是否值得？這必須用第 36 章的評測數據來回答，不能用感覺。

把這三項加回去之後，結論往往會反轉，或者變成「混合方案」：敏感資料走地端小模型，一般查詢走 API，並在兩者之間設一個路由器（第 33 章的程式 33B 就是在做這件事）。這才是多數臺灣單位的實際最佳解。

算帳的紀律

本書要求你在任何路線選擇的文件中，都必須同時列出：一次性投入、月變動成本、人力成本、建置期長度、品質差距、以及至少一個你沒有把握的假設。最後那一項最重要——把不確定的地方寫出來，比假裝什麼都算得準要專業得多。

1.4.5 一個給臺灣多數場域的務實建議

如果你現在必須立刻做決定，而且不確定該選哪一條，以下這個順序在多數臺灣的教育與中小企業場域是合理的起點：

1. 先用封閉 API 加上檢索，在兩週內做出可以給人試用的原型，目的是驗證「這個需求到底存不存在」。此時不要用真實敏感資料。
2. 同時建立你的評測集（第 36 章）。這一步最常被跳過，也最致命——沒有評測集，你之後所有的改進都只是感覺。
3. 原型被證明有價值之後，改用開放權重模型地端部署，比對品質差距。差距若在可接受範圍內，就留在地端。
4. 若品質不足，再進入適配階段：先試 LoRA（第 24 章），不夠再試持續預訓練（第 22 章）。
5. 自建預訓練請當成研究題目而不是產品路徑，除非你有國家級的算力與明確的主權需求。

1.4.6 三個臺灣單位的實際選擇（迷你案例）

以下三個案例經過改寫與去識別，但決策過程與結果都貼近真實。請在讀完之後回頭對照 1.4.2 的六個維度，看看你是否會做出相同的判斷。

案例一：中部某高中的作文回饋助理。需求是替國文老師產生初步的作文評語草稿，最後由老師修改後發給學生。資料是學生作品，屬於未成年人的個人資料。團隊最初打算用商業 API，但在法遵討論時發現：把學生作文原文送出去，即使供應商聲明不用於訓練，仍難以向家長說明。最後選擇單張 GPU 地端部署開放權重模型，品質略遜但可接受，且完全不必處理跨境傳輸的說明義務。這個案例的關鍵決策點是隱私，不是成本也不是能力——當硬約束存在時，其他維度的比較根本不會發生。

案例二：南部某精密機械廠的技術文件查詢。需求是讓現場人員用口語問「這台機器的主軸潤滑週期是多久」。團隊一開始想微調模型，投入兩個月後發現效果不佳——因為問題的答案就明明白白寫在文件裡，模型需要的不是「記住」而是「查到」。改成檢索增強之後兩週上線，正確率大幅提升，而且文件改版時不必重新訓練。這個案例的教訓是：先判斷任務屬於知識型還是能力型。知識型任務用檢索，能力型任務才需要微調，第 22 與 30 章會給出明確的判準。

案例三：某縣市政府的公文用語校對。需求是把承辦人擬的公文草稿檢查一遍，標出不符合公文格式與臺灣用語規範的地方。這個任務量大、單次價值低、且完全不涉及敏感判斷，因此選擇了成本最低的路線：小型開放權重模型加上規則式檢查表，模型只負責語意層面的建議，格式、稱謂、用語則由規則處理。這個案例提醒我們：不要把規則能解決的事丟給模型，那既貴又不可靠，而且無法解釋為什麼判定錯誤。

從三個案例抽出的三條判準

第一，先問這是知識型任務還是能力型任務——前者用檢索，後者才微調。第二，先問規則能不能解決其中一部分——能的話就交給規則。第三，隱私與法遵常常是唯一真正的硬約束，其他維度都可以取捨，這一項不行。這三條判準會分別在第 22、29 與 37 章被完整展開。

1.5 大模型生命週期：七個階段與一個迴圈

使用者看到的是一個輸入框和一段回覆。這一節要做的事，是把那個輸入框往回拆，一路拆到最上游。這張生命週期圖會在往後四十章反覆出現，本書的十篇結構基本上就是照著它排的。

1.5.1 七個階段

階段	關鍵決策	產出物	主要角色	章節
一、需求定義	使用者是誰、任務是什麼、什麼算失敗、誰負責	需求規格書、成功指標、不可接受風險清單	領域專家 + 工程師	1、40
二、資料與語	資料從哪來、授權是什麼、個資怎	語料清冊、Data Card、清理	資料工程 + 法務	6–9

階段	關鍵決策	產出物	主要角色	章節
料	麼處理、怎麼清理	管線、tokenizer		
三、預訓練	模型多大、訓練多久、算力哪裡來、怎麼避免發散	基礎模型權重、訓練日誌、實驗報告	訓練工程師	13–21
四、後訓練	指令資料怎麼寫、用 LoRA 還是全參數、怎麼對齊	指令模型、LoRA 權重、安全規範	模型工程師 + 標註團隊	22–27
五、評測	評什麼、誰來評、怎麼統計、題庫會不會外洩	評測集、盲評結果、紅隊報告	評測團隊 + 領域專家	36
六、部署	量化到幾位元、延遲目標多少、SLO 是什麼、怎麼回滾	服務端點、監控儀表板、維運手冊	系統 / MLOps 工程師	33、35
七、治理	誰能改、出事怎麼辦、要不要通報、多久稽核一次	治理文件、稽核軌跡、AI 影響評估	主管 + 法遵 + 工程	37

1.5.2 它不是一條直線，是一個迴圈

把上表當成瀑布式流程是最常見的錯誤。真實的專案長這樣：你在第五階段（評測）發現模型講不好臺灣的專有名詞，於是回到第二階段補語料、回到第四階段重新微調；你在第七階段（治理）收到一則資料退出請求，於是回到第二階段重整資料、可能還得回到第三階段重訓。

這個迴圈性質有一個直接的工程意涵：每個階段的產出物都必須是可版本化、可追溯、可重跑的。這就是為什麼第 4 章要花整整一章講可重現環境，為什麼第 8 章要建立 provenance ledger，為什麼第 16 章要求你記錄環境指紋與程式碼雜湊。這些東西在你第一次跑實驗時感覺很囉嗦，在你第三十次回頭找「當初到底是哪一版」時會救你一命。

1.5.3 成本分布：時間都花在哪裡

學生對大模型專案的想像通常是「大部分時間在訓練模型」。真實的時間分布比較接近下面這樣（依專案類型會有出入，但相對關係相當穩定）：

工作項目	時間占比	為什麼比想像中多
需求釐清與定義失敗邊界	約 10%	領域專家與工程師對「正確」的定義往往不一致，必須談到具體案例
資料取得、授權確認與清理	約 40%	取得管道、授權判讀、格式轉換、去識別、品質過濾、去重，每一步都

工作項目	時間占比	為什麼比想像中多
		會卡
建立評測集與評測流程	約 15%	出題、審題、標註、統計設計，而且必須防止外洩與污染
模型訓練與適配	約 15%	單次訓練可能只要幾小時，但要跑很多次才會對
部署、監控與維運	約 15%	延遲、成本、權限、回滾、告警，上線後才是長期戰
文件與治理	約 5%	通常被壓縮到最後才做，這是錯的，應該同步進行

請把這張表記牢。如果你的專題規劃書把 80% 的時間放在訓練，30% 的時間放在資料，那份規劃書幾乎一定會失敗。第 40 章的總整專題會強制你依照這個比例分配工時。

1.5.4 誰來做：一個大模型專案的角色分工

學生做專題時常常一個人扛下全部工作，這在學習階段是好事，但你必須知道真實專案是怎麼分工的，否則你會誤以為某些工作不存在。下表列出一個中型專案的典型角色，以及在大學專題中通常由誰兼任。

角色	負責什麼	最常見的失職	本書章節
領域專家	定義什麼是正確答案、什麼是不可接受的失敗、審核評測題目	只在期初出現一次，之後不參與	1、36、40
資料工程師	取得、清理、去識別、去重、版本化語料	沒有記錄來源，事後無法回答資料哪裡來	6–9
模型工程師	選底模、適配、微調、對齊、實驗紀錄	沒有 baseline 就開始調參	13–27
評測負責人	出題、盲評設計、統計、防止污染	用訓練資料裡的題目去測，量到的是背誦	36
系統 / MLOps	部署、監控、成本、回滾、SLO	沒有回滾方案就上線	33、35
法遵與倫理	授權判讀、個資、風險評估、通報流程	在系統快上線時才被找來，來不及改	8、37

在大學專題的四人小組中，一個可行的分配是：一人主責資料與法遵、一人主責模型、一人主責系統與部署、一人主責評測與文件，領域專家由指導老師或合作單位擔任。請注意「評測與文件」不是打

雜——那是最能決定專題成敗的角色，第 40 章的評分規準中有超過三分之一的配分落在這個人身上。

1.5.5 一個 18 週專題的真實時程長什麼樣

把 1.5.3 的成本分布落到具體的一個學期，會長成下面這樣。這份時程是給選修專題的學生看的，也是給第 40 章總整專題的預演。請注意第 1 到第 5 週完全沒有碰模型——這不是浪費時間，這是專案能不能成功的分水嶺。

週次	工作	產出物	常見的偏離
1-2 週	需求釐清、與領域專家訪談、定義不可接受的失敗	需求規格書 v0.1	跳過訪談，自己想像使用者要什麼
3-5 週	資料盤點、授權確認、取得管道建立、初步清理	語料清冊、Data Card 草案	先抓資料再想授權，事後無法補救
4-6 週	(與上並行) 建立評測集第一版並找人審題	30-60 題評測集	整學期都沒有評測集
6-8 週	baseline：用現成模型加提示與檢索，量測起點	baseline 報告	沒有 baseline 就開始微調
9-13 週	適配：LoRA 或持續預訓練，多輪實驗與消融	模型權重、實驗紀錄	只跑一個 seed 就下結論
12-15 週	部署：量化、服務化、監控、成本量測	可存取的服務與 SLO	在期末前一週才第一次部署
14-16 週	安全：紅隊測試、拒答檢查、權限驗證	紅隊報告	完全省略
16-18 週	文件、復盤、發表、開源或技轉決策	Model Card、技術報告、簡報	報告在最後三天趕出來

幾個時程上的經驗法則：資料階段的實際耗時通常是你估計的兩倍；第一次部署一定會失敗，所以要提前四週而不是提前一週；評測集一旦建立就不要再改，改了之前的數據就不能比較了。

1.6 臺灣本土大模型的必要性

這一節是全書的立論基礎。我要說服你的不是「愛臺灣所以要做臺灣模型」，而是「有五個可量測、可查證的技術理由，讓通用國際模型在臺灣場域會系統性地表現不足」。

1.6.1 理由一：用語不是小事

中文不是一種語言，是一組在不同社會裡各自演化的書面系統。臺灣的資訊科技用語與其他中文區的

差異，遠比一般人以為的大。當模型的訓練語料以其他中文區為主時，它會系統性地把不屬於這裡的詞當成預設值。

臺灣用語	其他中文區常見用語	臺灣用語	其他中文區常見用語
軟體 / 硬體	軟件 / 硬件	資料 / 資料庫	數據 / 數據庫
程式	程序	演算法	算法
網路	網絡	最佳化	優化
伺服器	服務器	記憶體	內存
介面	接口	硬碟	硬盤
專案	項目	影片	視頻
品質	質量	列印 / 印表機	打印 / 打印機
滑鼠	鼠標	解析度	分辨率
光碟	光盤	行動裝置	移動設備

在課堂上這也許只是彆扭；在公文、契約、規格書、法律文件裡，用錯詞就是實質的錯誤。而且這件事無法靠「請用臺灣用語回答」這句提示徹底解決——你可以在第 1A 實作中親自量測，提示能改善的比例有限，而且在長文本中會逐漸漂移回去。真正的解法在語料與後訓練，也就是第 22 與 23 章。

1.6.2 理由二：制度知識無法用翻譯補上

學測、統測、分科測驗、繁星推薦、免試入學超額比序、勞保與勞退的差別、健保的部分負擔、統一發票的載具、戶籍謄本與戶口名簿、統一編號與身分證字號的檢核規則、都市計畫的使用分區、勞基法的工時規範——這些不是詞彙問題，是制度問題。一個不曾大量閱讀臺灣文本的模型，不會有這些知識，而且它會用鄰近地區的類似制度去填補空白，填得非常順、非常自信、非常錯。

本章開頭那位老師遇到的正是這一類。而這一類錯誤最危險的地方在於：它看起來完全合理，只有真正懂的人才看得出來。這也意味著，臺灣本土評測集（第 36 章）不是加分項，是必需品。

1.6.3 理由三：token 稅是實實在在的成本

大多數主流 tokenizer 是以英文為中心設計的。同一段語意，英文可能只需要十幾個詞元，繁體中文卻需要兩三倍。這件事會沿著三條路徑轉成真金白銀的成本：

- API 費用按詞元計價，你付的錢比英文使用者多一到兩倍。
- 上下文長度以詞元計算，同樣的 32K 上下文，你能塞進去的實質內容比英文少一半以上。
- 訓練與推論的計算量隨序列長度增加，中文使用者的每一次互動都比較貴、比較慢。

第 7 章會讓你親手訓練 tokenizer 並量測這個比例，第 7C 實作會直接把 token fertility 換算成新臺幣。這是一個很好的教學時刻：學生會第一次感受到，語言的技術基礎設施如果不是為你設計的，代價會由你長期承擔。

1.6.4 理由四：資料主權與可控性

當一個學校、一家醫院、一間工廠把它的核心資料送進境外的模型服務，它同時交出了三件事：資料本身的控制權、服務可用性的控制權、以及模型行為變更的控制權。第三項最常被忽略——你依賴的模型可能在某天被更新、被下架、被調整安全策略，而你的系統會在毫無預警的情況下改變行為。

這不是要主張所有東西都必須地端。合理的做法是分級：把資料依敏感度分成幾層，最敏感的一層一定地端、中間層可用地端開放權重模型、最不敏感的一層可以用外部 API。第 37 章會教你怎麼把這個分級寫成可稽核的政策。

1.6.5 理由五：產業知識在臺灣人的手上

半導體製程、精密機械、工具機、電子代工、農漁業、醫療照護——這些領域的實務知識大量存在於臺灣的工單、SOP、技術報告、老師傅的口述裡，而且多半沒有上網。這些資料不會出現在任何國際模型的訓練語料中，任何人想做出這些領域真正有用的模型，都必須從臺灣的資料開始。

這是一個機會，也是一個責任。機會在於：這是別人做不到的差異化。責任在於：這些資料涉及營業秘密、個資與勞動關係，處理不當會造成真實傷害。第 8 章與第 37 章會把這件事講清楚。

1.6.6 把開場那個錯誤完整拆解一次

回到本章開頭：老師問「我們學校今年免試入學的超額比序怎麼算」，模型給了一套邏輯完整但屬於別的縣市的答案。這個錯誤看起來只有一個，實際上是四層問題疊在一起。學會拆解它，你就學會了本書的診斷方法。

層次	發生了什麼	根本成因	對應的修復手段
第一層	模型不知道「這所學校」是哪一所	提示中沒有提供學校身分，模型也無從得知	情境工程：把使用者身分帶入提示（第 29 章）

層次	發生了什麼	根本成因	對應的修復手段
第二層	模型不知道各縣市規則不同	訓練語料中臺灣升學制度的細節密度不足	補臺灣教育語料 (第 9、22 章)
第三層	模型用最常見的版本填補空白	訓練目標追求合理而非真實；缺乏拒答習慣	拒答訓練與不確定性表達 (第 25 章)
第四層	答案沒有出處，使用者無從查證	系統設計沒有要求引用	RAG 強制引用與來源顯示 (第 30、33 章)

注意這四層的修復成本差異極大。第一層改個提示就好，一小時內完成；第四層要建 RAG 系統，一到兩週；第二層要補語料再微調，數週到數月；第三層要重新設計後訓練資料，最貴。

工程判斷的關鍵在於：先做便宜的那幾層，量測還剩多少錯誤，再決定要不要往下走。很多團隊一開始就跳去做最貴的第三層（「我們來訓練自己的模型」），結果花了三個月才發現，其實加上第一層與第四層就已經解決了八成的問題。本書之所以把 RAG 排在第 30 章、把持續預訓練排在第 22 章，順序上看起來顛倒，正是因為在真實專案中你應該先試便宜的方案。

一個必須避免的錯誤結論

本節不是在說「國際模型很爛，我們自己做就好」。事實上本書大部分的實作都建立在開放權重的國際模型之上——那是務實且正確的做法。本節要說的是：把國際模型當成起點，而不是終點；差異化的價值在於我們往上加的那一層，而那一層只有我們做得出來。

1.7 臺灣的工程優勢與真實限制

談優勢容易變成口號，所以這一節同時談限制。學生必須看清楚我們站在哪一格，才能決定往哪裡走。

面向	臺灣的位置	對學生的意涵
半導體與硬體製造	先進製程、封裝測試與 AI 伺服器組裝在全球供應鏈中占關鍵位置	硬體與系統整合的實務知識容易取得；第 21 章會帶你盤點
邊緣運算與嵌入式	長期的工控、車電、機器人與嵌入式系統累積	小模型與邊緣部署是我們的強項戰場，第 26、35 章
製造場域資料	大量高價值、未數位化或未公開的產業知識	差異化的來源；但取得需要產學信任與資料治理，第 8 章

面向	臺灣的位置	對學生的意涵
算力	學術與中小企業可取得的 GPU 資源相對有限	必須擅長在受限算力下做事：LoRA、蒸餾、量化，第 24、26、35 章
語料	繁體中文高品質語料規模遠小於英文與簡體中文	語料工程本身就是研究題目，第 6–9 章
人才	工程訓練紮實，但頂尖 AI 人才競爭激烈且流動性高	教材必須讓學生真的會做，而不是只會用，這是本書的設計前提
資金與規模	單一機構難以支撐從零預訓練的資本支出	合作與開放共享比各自為政更有效，第 40 章鼓勵開源交付

1.7.1 三個可行的差異化切入點

光看優劣勢表容易停在感嘆。以下三個切入點是本書認為臺灣的大學實驗室與中小企業真正能做出成果的方向，每一個都對應到書中的具體章節，你在選擇貫穿式案例時可以參考。

1. 在地知識的深度而非廣度。不要試圖做一個什麼都會的臺灣模型，那需要的算力我們沒有。要做的是在某一個垂直領域做到別人做不到的深——工具機、模具、水產養殖、長照、地方文史都可以。關鍵在於資料取得的信任關係，那是外國團隊拿不到的（第 8、22、31 章）。
2. 邊緣與地端的工程能力。臺灣有大量不能上雲、不能斷線、必須低功耗的場域。把 3B 以下的模型調到在工控機上穩定跑一年不當機，這件事的技術含量與商業價值都被嚴重低估（第 26、35 章）。
3. 評測與治理的公共財。臺灣目前最缺的不是模型，是可信的臺灣評測集。一個做得夠嚴謹、夠公開、有審題紀錄與污染控制的臺灣題庫，它的影響力會超過任何單一模型，而且大學是最適合做這件事的機構（第 36 章）。

第三點值得多說一句。評測看起來不酷、不會登上新聞，但它決定了整個生態系能不能誠實地衡量進步。如果沒有人做這件事，所有人都只能靠感覺與行銷話術做決策。本書把第 36 章寫得比一般教材厚很多，就是這個原因。

把這張表濃縮成一句話：臺灣不太可能在「訓練最大的基礎模型」這件事上取得優勢，但完全有可能在「把基礎模型變成這裡真正能用的系統」這件事上做到世界級。本書的四十章就是照這個判斷排出來的——第三、四、五篇讓你懂原理與底層，第六篇之後的每一章都在教你怎麼把它變成這裡能用的東西。

1.8 本書地圖與學習建議

1.8.1 十篇四十章的能力遞進

本書分成十篇。第一到第五篇建立底層能力：你會親手寫出 tokenizer、注意力、Transformer 與一個可以生成繁體中文的 MiniFormosaGPT，並理解算力與分散式系統如何決定上限。第六篇之後轉向本土化與系統整合：適配、對齊、推理、檢索、智能體、應用開發、部署、評測、治理、多模態，最後以總整專題收束。

你不需要照順序讀完才能開始動手。事實上我建議你在讀完本章之後，立刻做兩件事：一是完成三個程式實作，二是選定一個你要跟著走完四十章的貫穿式案例（見規劃書的十二類案例）。有了具體的案例，後面每一章的內容都會自動找到落點。

1.8.2 四級硬體：沒有 GPU 也能學

層級	設備	本書中可完成的事	本章實作需求
A 級	一般筆電 (CPU)	tokenizer、注意力、MiniFormosaGPT tiny、API 應用、RAG、量化小模型推論	1A、1B、1C 全部可做
B 級	單張 GPU (8–24GB)	QLoRA 微調、7B 量化推論、Agent 與 VLM 實作、蒸餾	可額外加測地端模型
C 級	實驗室多卡叢集	持續預訓練、分散式訓練、尺度研究	—
D 級	國家級算力	從零預訓練基礎模型	—

1.8.3 三個常見的學習誤區

1. 只學提示工程。提示是推論期的技巧，天花板很低。真正的差異化在資料與後訓練，那是第二篇與第六篇的內容。
2. 跳過數學直接抄程式。第 3 章的張量形狀推理不難，但跳過它的人在第 11 章一定會卡住，而且卡住的時候不知道自己為什麼卡住。
3. 不建評測集就開始調整。沒有評測集，你所有的「感覺變好了」都不可信。第 36 章雖然排在後面，但評測集應該在專案第一週就開始建。

1.8.4 怎麼使用本書的三種讀者

如果你是大學生並修習這門課，請照章順序讀，每章的程式實作至少完成兩個，並從第一週就維護你

的需求規格書與評測集。

如果你是高中生或自學者，建議路徑是第 1–14 章（把 MiniFormosaGPT 做出來）、第 21 章（了解算力現實）、第 23–24 章（學會微調）、第 29–30 章（提示與 RAG）、第 32–34 章（Agent 與應用開發）、第 38–40 章（多模態與專題）。第 15–20 章與第 25–28 章可以先跳過數學推導只讀圖解，等你真的需要時再回來。

如果你是在職工程師或教師，建議從第 1、4、8 章建立基本判斷力，接著直接跳到你場域對應的章節，最後務必回頭讀第 36 與 37 章——評測與治理是最容易被跳過、也最容易在上線後付出代價的兩章。

1.9 本章名詞中英對照

本書採用臺灣慣用譯名，必要時保留英文原詞。完整術語表見附錄 K。

中文	英文	中文	英文
大語言模型	Large Language Model	基礎模型	Foundation Model
預訓練 / 後訓練	Pretraining / Post-training	指令微調	Instruction Tuning
詞元 / 分詞器	Token / Tokenizer	詞嵌入	Embedding
注意力機制	Attention	因果遮罩	Causal Mask
幻覺	Hallucination	對齊	Alignment
檢索增強生成	Retrieval-Augmented Generation	智能體	Agent
開放權重	Open Weights	量化	Quantization
尺度定律	Scaling Law	困惑度	Perplexity
災難性遺忘	Catastrophic Forgetting	模型編輯	Model Editing
資料溯源	Provenance	紅隊測試	Red Teaming

程式實作

本章三個實作都不需要 GPU，A 級筆電即可完成。程式碼、測試資料與預期輸出放在課程倉庫的 ch01/ 目錄下，執行前請先依第 4 章（或附錄 D）建立環境。

程式 1A 三模型繁中能力對照實驗

目標：用同一組臺灣情境提示測試三個模型，量測正確性、臺灣用語比例、token 數、延遲與成本，並輸出可比較的表格與雷達圖。這是在本書中的第一次「量測」，之後每一章都會用到同樣的紀律：先定義指標，再收集數據，最後才下結論。

ch01/lab1a_compare_models.py (節錄)

```
import time, json, csv, statistics
from pathlib import Path

# 臺灣用語檢核詞表：出現「其他中文區用語」即記為一次用語漂移
TW_TERMS = {
    "軟體": "軟件", "程式": "程序", "網路": "網絡", "資料": "數據",
    "伺服器": "服務器", "記憶體": "內存", "介面": "接口", "專案": "項目",
    "品質": "質量", "影片": "視頻", "演算法": "算法", "最佳化": "優化",
}

def term_drift(text: str):
    """回傳（漂移次數，命中的非臺灣用語清單）"""
    hits = [zh for tw, zh in TW_TERMS.items() if zh in text]
    return len(hits), hits

def run_case(client, model: str, prompt: str):
    t0 = time.perf_counter()
    resp = client.chat(model=model, prompt=prompt) # 由 adapters.py 統一介面
    latency = time.perf_counter() - t0
    drift, hits = term_drift(resp.text)
    return {
        "model": model,
        "latency_s": round(latency, 3),
        "in_tokens": resp.in_tokens,
        "out_tokens": resp.out_tokens,
        "cost_twd": round(resp.cost_twd, 4),
        "term_drift": drift,
        "drift_hits": hits,
        "text": resp.text,
    }

def main(models, prompts, out="results/ch01_compare.csv"):
    rows = []
    for m in models:
        for p in prompts:
            rows.append(run_case(CLIENT, m, p["prompt"]) | {"case_id": p["id"]})
    Path(out).parent.mkdir(parents=True, exist_ok=True)
    with open(out, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=rows[0].keys())
        w.writeheader(); w.writerows(rows)
    # 摘要：每個模型的中位延遲、平均輸出詞元、用語漂移率
    for m in models:
        sub = [r for r in rows if r["model"] == m]
        print(m,
              "延遲中位數", statistics.median(r["latency_s"] for r in sub),
```

```
"平均輸出 token", statistics.mean(r["out_tokens"] for r in sub),
"用語漂移率", sum(1 for r in sub if r["term_drift"] > 0) / len(sub))
```

提示集請至少包含六類，每類三題以上。下表列出每一類的設計意圖與範例，你必須自己改寫成你在縣市、學校與產業的版本——直接照抄會失去這個實驗的意義。

類別	範例題型 (請自行改寫)	設計意圖	主要觀察
制度題	說明學測、統測與分科測驗的差異與適用對象	測試臺灣教育制度知識密度	制度誤植
地理行政題	列出雲林縣所有的鄉鎮市並指出縣治所在	測試臺灣地理與行政區知識	幻覺、細節錯誤
法規誘發題	請說明某法規第 N 條的內容 (N 刻意超出實際條數)	主動誘發幻覺	是否編造、是否更正
產業術語題	解釋工具機的三軸與五軸差異，以及在臺灣的典型應用	測試產業知識與用語	用語漂移、術語錯誤
本土語言題	把一段華語句子寫成臺語漢字與臺羅拼音	測試低資源語言能力	拼寫錯誤、直接放棄
不可答題	詢問一個確定不存在的獎學金或校內單位	測試誠實拒答	拒答失敗

- 正確性請用人工標註，三人獨立評分後取多數；不要用模型評模型，那是第 36 章才會處理的進階議題，現在做會引入你還無法診斷的偏誤。
- 延遲請至少重複五次取中位數，並記錄測試時間與網路狀況。首次呼叫通常較慢，應該捨棄。
- 成本請換算成新臺幣，並同時記錄「回答同一題所需的 token 數」。你會發現有些模型雖然單價便宜，但因為輸出冗長，總成本反而較高。

延伸挑戰 (擇一完成)：

- 把同一組提示各跑五次，計算每個模型輸出的自我一致性 (五次回答彼此相同或等價的比例)。你會發現一致性低的題目，往往正是模型不確定的題目——這是第 27 章多樣本方法的直覺來源。
- 在提示前面加上一句「請使用臺灣繁體中文的慣用詞彙」，重測用語漂移率。記錄改善幅度，並特別觀察長回答的後半段是否漂移回去。這個實驗會讓你親身理解提示工程的天花板在哪裡。
- 把六類提示的正確率分開統計，畫出每個模型的能力雷達圖。你會看到「總分接近」的兩個模型，可能有完全不同的形狀——這正是為什麼單一總分是危險的。

程式 1B LLM 生命週期檢查器

目標：把 1.5 節那張七階段表變成一支可執行的工具。輸入一份用 YAML 描述的應用需求，輸出這個專案在每個階段必須完成的待辦事項、缺漏的欄位，以及風險等級。這支工具你會一路用到第 40 章的專題驗收。

ch01/spec_example.yaml

```
project: 校務與課程問答助理
owner: 智慧科技學院 AI 實驗室
users:
  - 在校學生
  - 導師與行政人員
tasks:
  - 查詢校規與學則
  - 查詢課程與選課規定
  - 查詢獎助學金申請期限
language: zh-TW
data:
  sources: [學則 PDF, 課程大綱, 行政公告]
  contains_pii: false
  license_checked: false      # 尚未確認 -> 檢查器會警告
unacceptable_failures:
  - 講錯申請截止日期
  - 捏造不存在的條號
metrics:
  - name: 引用正確率
    target: 0.98
  - name: 拒答率（無資料時）
    target: 0.90
deployment:
  location: on_premise
  expected_qps: 3
```

ch01/lab1b_lifecycle_check.py (節錄)

```
REQUIRED = {
    "需求定義": ["project", "users", "tasks", "unacceptable_failures", "metrics"],
    "資料與語料": ["data.sources", "data.license_checked", "data.contains_pii"],
    "部署": ["deployment.location", "deployment.expected_qps"],
}

RISK_RULES = [
    (lambda s: s["data"]["contains_pii"] and s["deployment"]["location"] != "on_premise",
     "高", "含個資卻規劃外部部署，須先完成法遵評估（第 8、37 章）"),
    (lambda s: not s["data"]["license_checked"],
     "高", "資料授權尚未確認，不得進入訓練階段（第 8 章）"),
    (lambda s: not s.get("unacceptable_failures"),
     "高", "未定義不可接受的失敗，無法設計評測與拒答策略（第 25、36 章）"),
    (lambda s: len(s.get("metrics", [])) < 2,
     "中", "指標少於兩項，建議至少涵蓋品質與安全兩個面向"),
]

def check(spec: dict):
    missing = [(stage, f) for stage, fields in REQUIRED.items()
               for f in fields if not _get(spec, f)]
    risks = [(lvl, msg) for rule, lvl, msg in RISK_RULES if rule(spec)]
    return missing, risks
```

延伸挑戰：把檢查器接上 CI，讓每次提交需求規格書時自動執行；並新增一條規則——若 `unacceptable_failures` 中出現醫療、法律或財務建議相關字樣，就強制標記為高風險並要求領域專家簽核。

程式 1C 臺灣常識三十題快測與錯誤分類

目標：建立本書第一個迷你評測集，並學會把錯誤分類。錯誤分類比正確率更重要——正確率告訴你「有多糟」，錯誤分類告訴你「該怎麼修」。

錯誤類型	判定方式	典型例子	對應的修復手段
用語漂移	出現非臺灣慣用詞	把「程式」寫成「程序」	後訓練資料與風格對齊（第 22–23 章）
制度誤植	套用其他地區制度	把升學管道講成其他學制	補臺灣語料 + 檢索（第 9、30 章）
幻覺	編造可查證但不存在的內容	虛構法條條號或校名	RAG 強制引用（第 30 章）
時效錯誤	內容曾經正確但已過期	引用已修正的舊條文	知識更新與模型編輯（第 28 章）
拒答失敗	無資料時仍給出答案	對明知無解的題目硬答	拒答訓練（第 25 章）
過度拒答	有資料卻拒絕回答	把一般校務問題當成敏感題	安全規範校準（第 25、36 章）

ch01/lab1c_taiwan_quiz.py (節錄)

```
# quiz.jsonl 每行一題：
# {"id": "TW-007", "q": "...", "gold": "...", "type": "制度", "answerable": true}

def classify_error(pred: str, item: dict) -> str:
    if not item["answerable"]:
        return "OK" if is_refusal(pred) else "拒答失敗"
    if is_refusal(pred):
        return "過度拒答"
    if match_gold(pred, item["gold"]):
        return "OK"
    if term_drift(pred)[0] > 0:
        return "用語漂移"
    if cites_nonexistent(pred):
        return "幻覺"
    if matches_outdated(pred, item.get("outdated_gold")):
        return "時效錯誤"
    return "制度誤植"

def report(results):
    from collections import Counter
    c = Counter(r["error"] for r in results)
    total = len(results)
```

```
for k, v in c.most_common():
    print(f"{k:8s} {v:3d} {v/total:6.1%}")
```

三十題的分布建議如下，並在 jsonl 中以 type 欄位標記，這樣你才能算出「哪一類最弱」而不只是一個總分。

類別	題數	出題來源建議	標準答案的出處要求
教育制度	6	教育部與各校公開簡章、招生規定	須附文件名稱、版本與頁碼
地理與行政	5	內政部與地方政府官方網站	須附網址與查核日期
法規與公文	5	全國法規資料庫、機關公告	須附法規名稱、條號與修正日期
產業與技術	6	公協會資料、廠商規格書、技術報告	須附來源文件
本土語言與文化	4	教育部辭典、地方文史資料	須附辭典條目
不可答 (誠實測試)	4	自行設計，確認確實不存在	須說明為何不存在

出題原則

三十題請自己出，不要從網路上抓現成題目——現成題目很可能已經在模型的訓練資料裡，你量到的會是背誦而不是能力。這個問題在第 36 章叫做 benchmark 污染，是評測工程最核心的難題之一。另外，題目必須有明確、可查證的標準答案與出處，出處要寫進 jsonl。

系統案例：本土模型與通用模型的臺灣情境對照

本章的系統案例不寫程式，而是設計一場實驗，並把它寫成一份可被同儕檢驗的報告。你要比較的是：臺灣本土訓練或適配的模型（例如 TAIDE、Taiwan LLM 一類），與國際通用開放權重模型，在臺灣公文、校務規章與電子製造詞彙上的表現差異。

實驗設計

1. 受測對象：至少一個臺灣本土模型、一個國際開放權重模型（規模相近）、一個商業封閉模型作為上界參考。記錄每個模型的版本、取得日期與授權。
2. 任務集：三個領域各 20 題，共 60 題。公文（用語、格式、法規引用）、校務（學則、選課、獎助）、電子製造（料號、設備術語、中英混寫）。
3. 控制變因：同一組提示、同一組解碼參數（temperature 固定為 0.2）、同一位標註者順序隨機、

盲評（標註者不知道哪個回答來自哪個模型）。

4. 量測指標：正確率、用語漂移率、幻覺率、平均輸出 token 數、每題成本、中位延遲。
5. 歸因分析：對每一個錯誤，判斷成因屬於「語料缺乏」「tokenizer 不利」還是「對齊不足」三者之一。這是本案例最重要的一步。

你應該會觀察到的現象（先預測，再驗證）

- 本土模型在用語與制度題上的優勢通常明顯，但在一般推理與程式題上可能落後於規模相近的國際模型。
- 規模較大的國際模型在許多題目上仍可能贏過小型本土模型——這不矛盾，這正是第 22 章要教你的：把本土適配加在更強的底模上。
- 同一題的成本差異可能達到數倍，主要來自 tokenizer 效率與輸出長度，而不是單價。

報告格式（本書所有系統案例共用）

這份格式你會用四十次，請從第一次就寫對。每一節都有它存在的理由，不要為了篇幅而刪。

章節	要寫什麼	常見錯誤	篇幅
一、問題與範圍	誰會用、解決什麼問題、明確排除什麼	寫得太大，變成無法驗收的願景	半頁
二、方法	受測對象、任務集、控制變因、指標定義	沒寫模型版本與取得日期	一頁
三、結果	表格與圖，含樣本數與變異	只放平均值，不放分布	一至二頁
四、歸因分析	每一類錯誤的成因判斷與證據	直接跳到結論，沒有歸因	一頁
五、限制	哪些結論不成立、哪些沒測到	整節省略（最嚴重的錯誤）	半頁
六、建議	下一步該做什麼，以及成本估計	建議與數據無關	半頁

請注意：在寫結論時，不可以只說「A 模型比 B 模型好」。你必須寫成「在這 60 題、這組指標、這個時間點、這個版本下，A 在制度題上高出 B 多少個百分點，信賴區間為何」。這種寫法看起來囉嗦，但它是唯一誠實的寫法，也是第 16 與 36 章會反覆訓練的表達紀律。

章末評量

一、概念題

1. 用自己的話說明「基礎模型」與「大語言模型」的差別，並各舉一個不屬於對方的例子。
2. 為什麼說「人工智慧不等於機器學習」？舉出兩個屬於 AI 但不屬於 ML 的方法，並說明它們在現代大模型系統中仍有什麼用途。
3. 解釋為什麼「下一個詞元預測」這個看似簡單的目標，可以讓模型學會翻譯與寫程式。這個說法有什麼極限？
4. 指令模型與基礎模型的行為差異是什麼？造成差異的是哪一個訓練階段？

二、推導與計算題

1. 某任務的中文提示平均 800 個詞元、回覆 600 個詞元，每日 5,000 次呼叫。若輸入單價為每千詞元 0.9 元、輸出為 3.6 元（新臺幣），計算月成本。若改用 tokenizer 效率高 35% 的模型，月成本變成多少？
2. 承上題，若地端部署需一次性投入 45 萬元硬體、每月電力與維運 1.2 萬元，計算損益兩平所需的月數（不計折舊與人力）。並說明有哪三項成本是這個計算沒有涵蓋的。
3. 某模型上下文長度為 32K 詞元。若繁體中文的 token fertility 是英文的 2.3 倍，估算中文使用者實際能放入的字數大約相當於英文的多少比例。

三、除錯題

1. 同學的 1A 實驗得到「A 模型延遲只有 B 模型的三分之一」的結論，但他每個模型只跑了一次，而且 A 模型是第二個測的。指出這個實驗至少三個設計缺陷，並說明如何修正。
2. 某系統上線後，使用者回報「模型會編造不存在的獎學金名稱」。請依 1.3.6 的檢查表判斷這屬於哪一種失效模式，並依成本由低到高排出三個處置方案。
3. 一份需求規格書的 unacceptable_failures 欄位寫著「回答不夠好」。說明為什麼這個寫法無法使用，並改寫成兩條可驗收的敘述。

四、系統設計題

1. 為某高中設計一個「課程與升學諮詢助理」。請畫出七階段生命週期，標明每階段的產出物、負責人、以及你認為最容易被跳過的一步。特別說明未成年人資料要如何處理。
2. 某中小型工具機廠希望導入維修知識助理，製程參數不得外流，預算有限，資訊人員僅一人。請從四條路線中選一條並論證，需涵蓋控制力、成本、隱私、授權、可維護性、能力上限六個維

度。

五、臺灣情境題

1. 找出三個你自己專業領域中的臺灣專有名詞或制度，測試至少兩個模型能否正確說明，記錄錯誤並依 1C 的分類法歸因。
2. 從政府公開資料平台挑一份資料集，判斷它的授權條款是否允許用於模型訓練，並寫出你的判斷依據與不確定之處（不確定的部分請明確標示需要法律確認）。

六、倫理討論題

1. 若一所學校的 AI 助理曾經給出錯誤的獎學金申請期限，導致學生錯過申請，責任應如何分配？學校、系統開發者、模型提供者、使用者各自的角色是什麼？請提出一套事前可降低此風險的機制。
2. 有人主張「臺灣語料太少，直接用其他地區的中文語料訓練就好，反正字都看得懂」。請提出至少三個技術上的反駁，以及一個你認為這個主張有道理的地方。
3. 訓練本土模型需要大量臺灣文本，其中可能包含個人的公開發言、學生作品與新聞報導。在「保存本土語言文化」與「個人對自己文字的控制權」之間，你認為應該如何取得平衡？請設計一套退出機制。

評量提示（給自學者）

- 推導題第 1 題：先算單次成本 = 輸入詞元數 ÷ 1000 × 單價 + 輸出詞元數 ÷ 1000 × 單價，再乘以每日次數與 30 天。tokenizer 效率提升 35% 意味著同樣內容的詞元數變成原本的約 74% ($1 \div 1.35$)，兩項單價都受影響。
- 推導題第 2 題：損益兩平月數 = 一次性投入 ÷ (API 月成本 - 地端月成本)。沒涵蓋的成本至少包括人力、機會成本與品質差距，另可再考慮硬體折舊與電費波動。
- 除錯題第 1 題：三個缺陷分別是樣本數不足（只跑一次）、順序效應未控制（暖機與快取）、以及沒有記錄網路狀況與時間點。修正方式是重複五次以上取中位數、隨機化順序、捨棄第一次呼叫。
- 系統設計題第 2 題：關鍵是「製程參數不得外流」直接排除封閉 API 路線，而「資訊人員僅一人」則使可維護性成為最重的權重，因此答案通常是「開放權重 + RAG，暫不微調」，而不是自建訓練。

研究延伸與版本注意事項

- 基礎模型概念的原始論述：史丹佛 CRFM 團隊 2021 年關於基礎模型機會與風險的長篇報告，建議至少讀第一章與倫理章節。
- Transformer 原始論文 (2017) 與 GPT 系列技術報告：本章只講概念，數學細節在第 11–13 章處理，建議讀完第 13 章後再回頭讀原論文。
- 尺度定律相關研究 (Kaplan 等人 2020、Hoffmann 等人 2022)：第 17 章會完整處理，本章只需知道「參數、資料、算力有最適配比」。
- 課程資源：Stanford CS336 (從零建構語言模型)、CMU 進階自然語言處理與 LLM 系統、UC Berkeley LLM Agents、李宏毅教授的生成式 AI 課程影片。前三者提供作業與講義，後者以中文講授、適合作為本書的影音補充。
- 開源教材：浙江大學《大模型基礎》(Foundations of LLMs) 可免費取得完整 PDF 與論文清單，適合作為本書第七篇 (Prompt、PEFT、模型編輯、RAG) 的延伸閱讀。
- 實作書：Raschka 的《Build a Large Language Model (From Scratch)》與其推理模型續作，本書第三、四篇的實作路線與其高度相容，可互為補充。
- 臺灣資源：TAIDE 計畫、Taiwan LLM 系列、國家高速網路與計算中心的算力申請說明、政府資料開放平台的授權條款。這些資源的狀態與條款變動較快，請於使用前查核官方頁面並記錄查核日期。

版本敏感內容的處理原則

本章提到的所有模型名稱、價格、上下文長度、授權條款都會變動。正文只保留穩定的原理與方法；具體數字一律以課程網頁的「線上補充」頁面為準，並在每次開課前重新查核。你在做任何實驗報告時，也請養成同樣的習慣：在報告開頭寫明模型版本、取得日期與查核日期。這不是形式主義，這是讓你的實驗在半年後仍然可被理解的唯一方法。

常見問題

以下是歷屆學生在第一週最常提出的問題，答案都指向後續章節，可以當成閱讀導引。

問：我沒有 GPU，是不是就不用學了？答：本章的三個實作、第 3、4、6、7、10、11 章的全部實作，以及第 29 到 34 章的大部分實作，都只需要一般筆電。真正需要 GPU 的是第 15、19、22 到 26 章的訓練實驗，而這些章節都提供了縮小版設定與可下載的中間產物，讓你在 CPU 上也能走完流程、看懂每一步。學不學得會取決於你有沒有動手，不取決於你有沒有卡。

問：既然商業模型這麼強，我們自己做的意義是什麼？答：兩個意義。第一是能力上的：你只有自己做過一次，才有能力判斷別人的模型好不好、貴不貴、值不值得信。第二是結構上的：本書 1.6 節列

出的五個理由——用語、制度、token 稅、資料主權、產業知識——不會因為國際模型變強而消失，反而會因為依賴加深而更重要。

問：我該先學哪一個框架？答：不要以框架為單位學習。本書的原則是先手寫一次再用函式庫，因為函式庫每年都在換，而你手寫過的那個注意力機制不會過期。第 4 章會建立環境，但你在第 11 到 13 章寫的程式碼幾乎不依賴任何高階框架。

問：大模型會不會取代我的工作？答：這個問題超出本書的範圍，但有一個工程上的觀察值得分享：目前大模型最不擅長的，恰恰是本書反覆強調的那些事——定義問題、判斷什麼是不可接受的失敗、確認資料來源是否合法、決定要不要相信一個結果、以及承擔責任。這門課要訓練的正是這些能力。

問：本書提到的模型與工具會不會很快就過時？答：具體的名稱會，方法不會。分詞、注意力、訓練迴圈、尺度定律、評測設計、資料治理這些東西的半衰期遠比任何一個模型長。這也是為什麼本書刻意把版本敏感的內容全部移到線上補充頁面。

教師使用建議

本章排在第 1 週，建議 2 小時講授加 1 小時實驗。以下是實際授課後整理出的節奏建議，供採用本書的教師參考。

時段	內容	互動設計	注意事項
前 20 分鐘	開場案例與 1.1 節名詞界定	請學生當場問模型一個本地問題並唸出答案	事先準備好備用案例，以免當天模型剛好答對
中間 40 分鐘	1.2 與 1.3 節，能力與失效模式	分組找出各自領域的失效案例並上台一分鐘報告	這是全章最重要的一段，不要為了趕進度壓縮
後 30 分鐘	1.4 與 1.5 節，路線與生命週期	以真實需求做路線選擇的小組辯論	避免變成品牌之爭，聚焦六個維度
最後 30 分鐘	1.6 至 1.8 節與作業說明	當場選定貫穿式案例並登記	務必當場完成選案，不要拖到第二週
實驗課 1 小時	程式 1A 與環境檢查	兩人一組，一人跑實驗一人記錄	API 金鑰請以課程共用帳號發放並設用量上限

高中授課的調整：1.1 節的表格只講前四列，1.4 與 1.5 節改為看表格與討論，不講細節；把省下的時間全部給 1.3 節的失效模式與實作 1A。高中生對「模型會自信地講錯」這件事的反應通常非常強烈，

那是最好的教學時刻。

本章小結

1. 大模型是基礎模型在文字模態上的具體化。「大」指的是參數、資料與算力三者同時放大，而三者之間有最適配比。
2. 大模型的能力來自三個層次：預訓練決定天花板、後訓練決定行為、推論期決定這一次回答的品質。臺灣的施力點主要在後兩者。
3. 五種失效模式——幻覺、知識時效、偏見、脆弱推理、過度自信——各有明確成因與對應的工程對策，不能靠「提示寫好一點」解決。
4. 四條技術路線的選擇必須同時評估控制力、成本、隱私、授權、可維護性與能力上限；「開放權重」不等於「開源」，授權條款必須逐條確認。
5. 生命週期是一個迴圈而不是一條直線，因此每個階段的產出物都必須可版本化、可追溯、可重跑。時間的大宗花在資料與評測，不是訓練。
6. 臺灣需要本土模型有五個可量測的技術理由：用語、制度知識、token 稅、資料主權、產業知識。這不是情感訴求，是工程判斷。
7. 臺灣的優勢在硬體、邊緣運算、製造場域資料與工程人才；限制在算力、語料與資金規模。合理的策略是站在開放權重模型之上做出差異化，而不是從零開始比大。

成果檢核

完成本章後，你必須繳交一份「臺灣本土大模型需求規格書 v0.1」，格式參照程式 1B 的 YAML 範本，並通過檢查器的驗證。規格書至少包含：

- 使用者：具體到角色與情境，不能只寫「一般民眾」。
- 任務清單：每項任務要能被驗收，寫成「使用者輸入什麼、系統輸出什麼、怎麼判定正確」。
- 語言與用語規範：明確指定臺灣繁體中文，並列出必須正確使用的專業術語。
- 資料邊界：哪些資料可用、哪些不可用、哪些需要進一步確認授權。
- 評測指標：至少兩項，一項品質、一項安全，並各自寫出目標值。
- 不可接受的失敗：至少兩條，且必須是可觀察、可驗收的敘述。
- 本章三個程式實作的執行結果與一頁分析。

這份規格書會在第 9、18、27、36 週的四次階段評量中反覆更新。到第 40 章的總整專題時，它會成為你專題文件的第一節。現在寫得越具體，未來三十九章你就越知道自己在做什麼。

